

Министерство образования Республики Беларусь
Учреждение образования
«Брестский государственный технический университет»
Кафедра ИИТ

Лабораторная работа №5
По дисциплине: «ОМО»
Тема: «Нелинейные ИНС в задачах регрессии»

Выполнил:
Студент 3-го курса
Группы АС-66
Осовец А.О.
Проверил:
Крощенко А.А.

Брест 2025

Вариант 7

Цель: исследовать работу нелинейной ИНС в задаче регрессии, обучив модель аппроксимировать заданную нелинейную функцию и оценив качество прогнозирования.

Задание:

1. Выполнить моделирование прогнозирующей нелинейной ИНС. Для генерации обучающих и тестовых данных использовать функцию `target_function`. Для прогнозирования использовать многослойную ИНС с одним скрытым слоем. В качестве функций активации для скрытого слоя использовать сигмоидную функцию, для выходного - линейную.

$$y = a \cos(bx) + c \sin(dx)$$

7	0.3	0.1	0.06	0.1	6	2
---	-----	-----	------	-----	---	---

```
import torch
import torch.nn as nn
import torch.optim as optim
import matplotlib.pyplot as plt
import pandas as pd

# 1. Генерация обучающих и тестовых данных
def target_function(x, a=0.2, b=0.4, c=0.09, d=0.4):
    return a * torch.cos(b * x) + c * torch.sin(d * x)

a, b, c, d = 0.2, 0.4, 0.09, 0.4
input_size = 6
hidden_size = 2
num_samples = 200
train_ratio = 0.8

x = torch.linspace(0, 200, num_samples).reshape(-1, 1)
y = target_function(x, a, b, c, d)

X = torch.cat([y[i:num_samples - input_size + i] for i in range(input_size)], dim=1)
y_target = y[input_size:]

split_idx = int(X.shape[0] * train_ratio)
X_train, X_test = X[:split_idx], X[split_idx:]
y_train, y_test = y_target[:split_idx], y_target[split_idx:]

print("Данные сгенерированы для функции y = a*cos(bx) + c*sin(dx)")
print(f"Параметры: a={a}, b={b}, c={c}, d={d}")
print(f"Размер обучающей выборки: {X_train.shape}, тестовой: {X_test.shape}")

# 2. Архитектура ИНС
class NonlinearRegressor(nn.Module):
    def __init__(self, input_dim, hidden_dim):
        super().__init__()
        self.hidden = nn.Sequential(
            nn.Linear(input_dim, hidden_dim),
            nn.Sigmoid()
        )
        self.output = nn.Linear(hidden_dim, 1)
```

```

def forward(self, x):
    x = self.hidden(x)
    return self.output(x)

model = NonlinearRegressor(input_size, hidden_size)
criterion = nn.MSELoss()
optimizer = optim.Adam(model.parameters(), lr=0.01)

# 3. Обучение модели
losses = []
epochs = 5000

for epoch in range(epochs):
    model.train()
    y_pred = model(X_train)
    loss = criterion(y_pred, y_train)
    optimizer.zero_grad()
    loss.backward()
    optimizer.step()
    losses.append(loss.item())

print("\nОбучение завершено")
print(f"Минимальная ошибка достигнута при a={a}, финальный MSE={min(losses):.6f}")

# 4. График прогнозируемой функции на обучающем участке
model.eval()
with torch.no_grad():
    y_train_pred = model(X_train)

plt.figure(figsize=(10, 4))
plt.plot(x[:split_idx], y_train, label="Эталон")
plt.plot(x[:split_idx], y_train_pred, label="Прогноз")
plt.title("Прогнозируемая функция на обучающем участке")
plt.xlabel("x")
plt.ylabel("y")
plt.legend()
plt.grid(True)
plt.show()

# 5. Результаты обучения: таблица и график ошибки
train_table = pd.DataFrame({
    "Эталонное значение": y_train.squeeze().numpy(),
    "Полученное значение": y_train_pred.squeeze().numpy(),
    "Отклонение": (y_train_pred - y_train).squeeze().numpy()
})
print("\nРезультаты обучения (первые строки):")
print(train_table.head())

plt.figure(figsize=(10, 4))
plt.plot(losses)
plt.title("График изменения ошибки по эпохам")
plt.xlabel("Эпоха")
plt.ylabel("MSE")
plt.grid(True)
plt.show()

# 6. Результаты прогнозирования
with torch.no_grad():
    y_test_pred = model(X_test)
    test_table = pd.DataFrame({
        "Эталонное значение": y_test.squeeze().numpy(),
        "Полученное значение": y_test_pred.squeeze().numpy(),
        "Отклонение": (y_test_pred - y_test).squeeze().numpy()
    })

```

```
print("\nРезультаты прогнозирования (первые строки):")
print(test_table.head())
```

Результат:

Данные сгенерированы для функции $y = a \cdot \cos(bx) + c \cdot \sin(dx)$
Параметры: $a=0.2$, $b=0.4$, $c=0.09$, $d=0.4$
Размер обучающей выборки: `torch.Size([155, 6])`, тестовой: `torch.Size([39, 6])`

Обучение завершено
Минимальная ошибка достигнута при $a=0.2$, финальный MSE=0.000010
2025-12-02 10:38:04.686 Python[25796:927314] +[IMKClient subclass]: chose IMKClient_Legacy
2025-12-02 10:38:04.686 Python[25796:927314] +[IMKInputSession subclass]: chose IMKInputSession_Legacy
2025-12-02 10:38:07.432 Python[25796:927314] The class 'NSSavePanel' overrides the method identifier. This method is implemented by class 'NSWindow'

Результаты обучения (первые строки):

	Эталонное значение	Полученное значение	Отклонение
0	-0.089110	-0.092945	-0.003835
1	-0.160416	-0.162940	-0.002524
2	-0.206143	-0.204142	0.002001
3	-0.219002	-0.215096	0.003906
4	-0.196941	-0.196046	0.000895

Результаты прогнозирования (первые строки):

	Эталонное значение	Полученное значение	Отклонение
0	0.022305	0.024099	0.001794
1	-0.064840	-0.067922	-0.003083
2	-0.141646	-0.145127	-0.003480
3	-0.195868	-0.195174	0.000694
4	-0.218859	-0.214987	0.003872

Process finished with exit code 0

